

Student and College Management System

Campus Ledger

Golamari Shashanth Reddy

25WU0101041

Hansini Baggu

25WU0101045

Kandibanda Balaji

25WU0101057

Problem → ER & schema → MySQL & queries → working UI

Problem, objectives and users

Problem

Admissions, teaching, examinations and accounts maintain related student records. Separate registers repeat data and slow reconciliation.

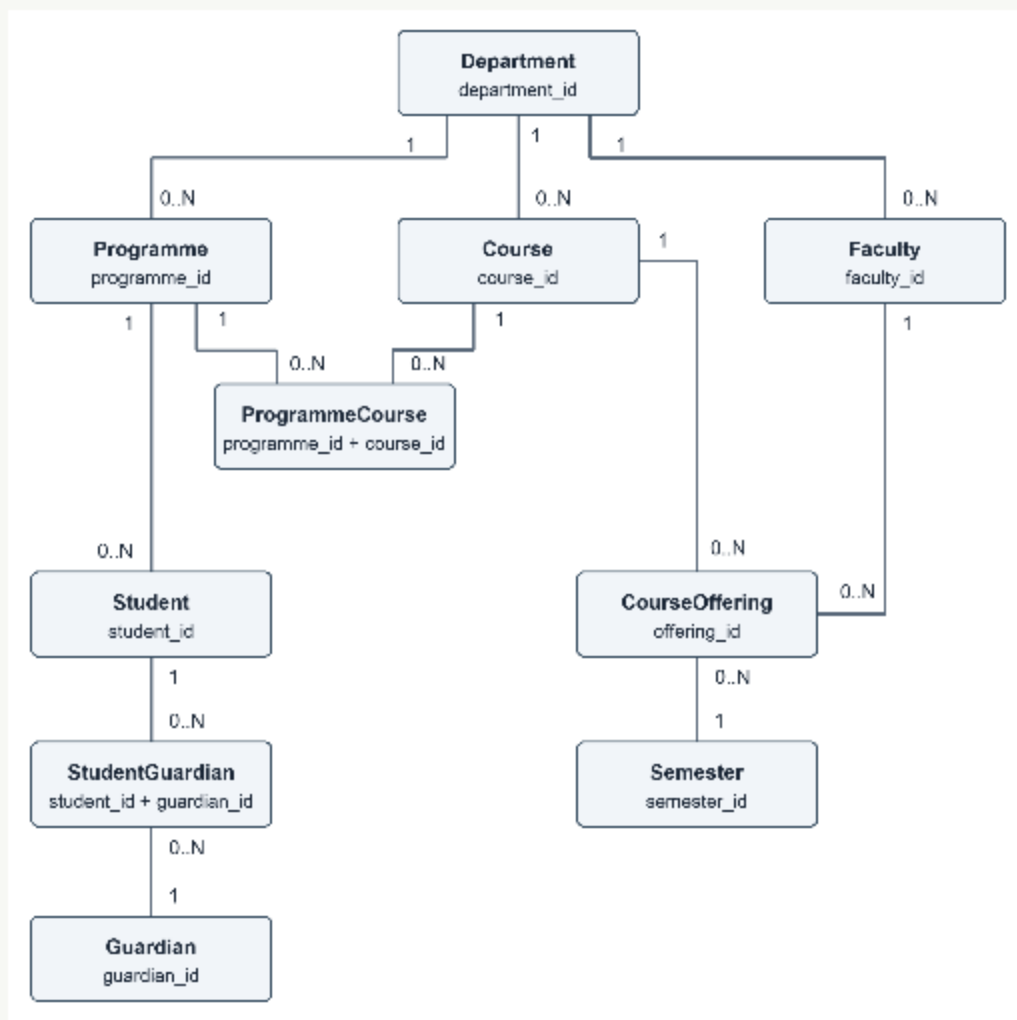
Objective

One connected database for admission, registration, attendance, grades, bills, payments and reports.

Users

Admin • Faculty • Examinations • Accounts • Student • Head of department

ER model: academic structure



Core entities

Department • Programme
 Course • Faculty • Semester
 Student • Guardian
 Course Offering • Registration

Keys and relationships

Each entity has a primary key. Foreign keys connect the child to exactly one parent; a parent can have zero or many children.

Relationships, cardinality and participation

Relationship	Cardinality	Child participation
Department → Programme	1 to 0..N	Programme requires one department
Student → Registration	1 to 0..N	Registration requires one student
Offering → Registration	1 to 0..N	Registration requires one offering
Registration → Attendance	1 to 0..N	Attendance requires a registration
Fee Bill → Payment	1 to 0..N	Payment requires one bill

Bridge tables resolve many-to-many links: StudentGuardian, ProgrammeCourse and Registration.

Relational schema and Third Normal Form

Representative keys

Student(student_id PK, programme_id FK, registration_no UNIQUE)

CourseOffering(offering_id PK, course_id FK, semester_id FK, faculty_id FK)

Registration(registration_id PK, student_id FK, offering_id FK)

Attendance(registration_id + session_id composite PK)

FeeBill(bill_id PK, student_id FK, semester_id FK)

Payment(payment_id PK, bill_id FK)

Why 3NF?

Each table stores one subject. Repeating values become separate rows; balances and attendance percentages are derived through views.

MySQL creation, constraints and sample rows

Live file: DBMS_Course_Project_All_Commands.sql

```
CREATE DATABASE college_pbl_presentation;  
CREATE TABLE registration (  
  registration_id INT AUTO_INCREMENT PRIMARY KEY,  
  student_id INT NOT NULL, offering_id INT NOT NULL,  
  UNIQUE (student_id, offering_id),  
  FOREIGN KEY (student_id) REFERENCES student(student_id),  
  FOREIGN KEY (offering_id) REFERENCES  
course_offering(offering_id)  
);
```

Verified on MySQL 8

21 base tables
4 reporting views
5+ rows in every table
45 passing tests

Real run output is included as SQL_Execution_Output.txt.

SELECT joins and aggregate outputs

Join

Student → Registration → CourseOffering → Course → Semester

Example: 2026CS001, Aarav Sharma, CS301, section A, 2026–27 Odd

Aggregate: active students by section

CS301 A	9
CS301 B	0
CS302 A	9
CS303 A	0
EC301 A	3

COUNT + LEFT JOIN includes sections with zero registrations.

Full queries and actual outputs are included in the repository.

Working UI: view and insert

Before: five departments

The screenshot shows the 'Departments' page in the Campus Ledger application. The page has a dark sidebar with navigation links and a main content area. The main content area includes a 'COLLEGE SETUP' header, a 'Departments' title, and a description: 'Maintain the reference records that connect college operations.' Below this is an 'Add record' form with two input fields: 'Code' and 'Department name', and a 'Save record' button. At the bottom is a table listing five departments.

Code	Department name	Action
BO	Biological Sciences	Edit Delete
CE	Civil Engineering	Edit Delete
ME	Mechanical Engineering	Edit Delete
EE	Electrical and Communication	Edit Delete
CS	Computer Science and Engineering	Edit Delete

After UI insert: six departments

The screenshot shows the 'Departments' page after a new department has been added. The 'Add record' form now has a green 'Department saved' message above it. The table at the bottom now lists six departments, including a new one: 'Temporary concentration department'.

Code	Department name	Action
DBAC-04	Temporary concentration department	Edit Delete
BO	Biological Sciences	Edit Delete
CE	Civil Engineering	Edit Delete
ME	Mechanical Engineering	Edit Delete
EE	Electrical and Communication	Edit Delete
CS	Computer Science and Engineering	Edit Delete

POST create → INSERT INTO department → SELECT confirms DEMO-QA

Working UI: delete and database reflection

Before delete: DEMO-QA is present

Campus Ledger
Account & college management

COLLEGE OPERATIONS [View Demo](#)

Departments send

COLLEGE SETUP

Departments

Maintain the reference records that connect college operations.

Add record

Code: Department name:

[Save record](#)

Code	Department name	Action
DEMO-QA	Temporary demonstration department	Edit Delete
SD	Biological Sciences	Edit Delete
CE	Civil Engineering	Edit Delete
ME	Mechanical Engineering	Edit Delete
EE	Electrical & Electronic Engineering	Edit Delete
CS	Computer Science and Engineering	Edit Delete

Campus Ledger - DBMS Project V1 | © 2024 | All rights reserved. | Page 1 of 1

After delete: DEMO-QA is absent

Campus Ledger
Account & college management

COLLEGE OPERATIONS [View Demo](#)

Departments send

COLLEGE SETUP

Departments

Maintain the reference records that connect college operations.

Add record

Code: Department name:

[Save record](#)

Code	Department name	Action
SD	Biological Sciences	Edit Delete
CE	Civil Engineering	Edit Delete
ME	Mechanical Engineering	Edit Delete
EE	Electrical & Electronic Engineering	Edit Delete
CS	Computer Science and Engineering	Edit Delete

Campus Ledger - DBMS Project V1 | © 2024 | All rights reserved. | Page 1 of 1

POST delete → DELETE FROM department → SELECT returns zero rows

Results, limits and questions

Verified result

45 tests passed · 0 failures · 0 errors

What the solution supports

Admissions, section registration, attendance, exam results, grades, fee collection and six CSV reports.

Scope of this prototype

Synthetic records. The 75% attendance threshold and grade scale are demo rules. The app runs locally.